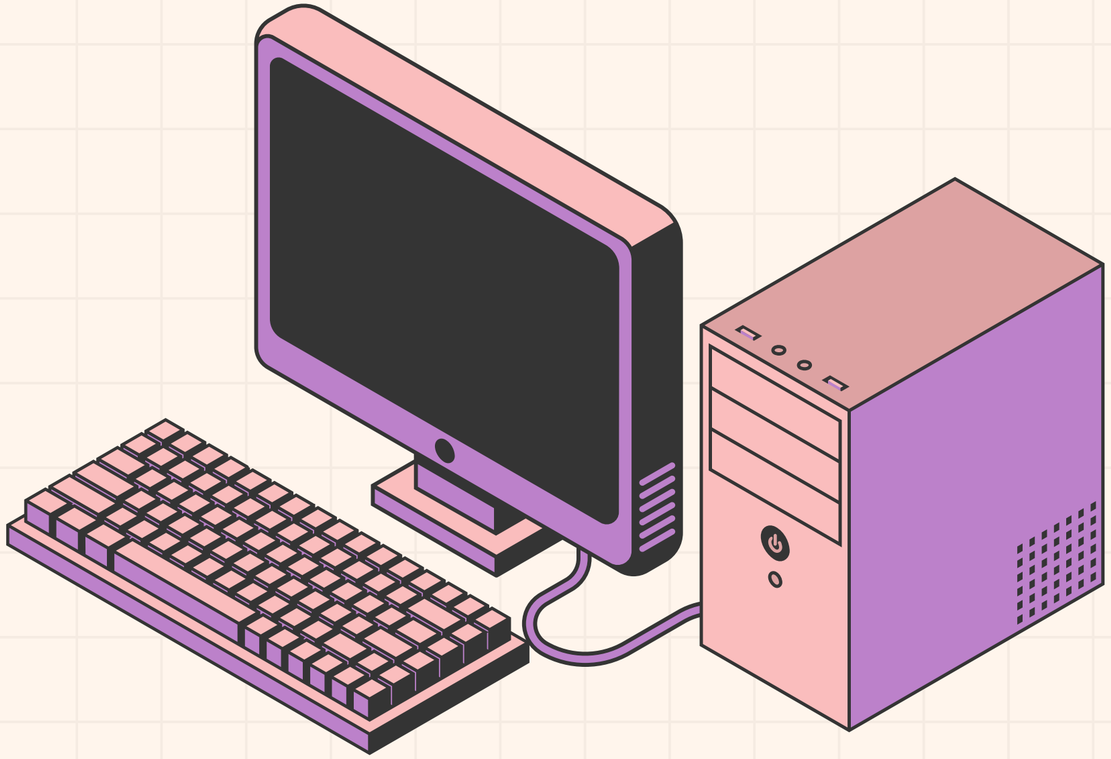
CÓDIGO LIMPIO

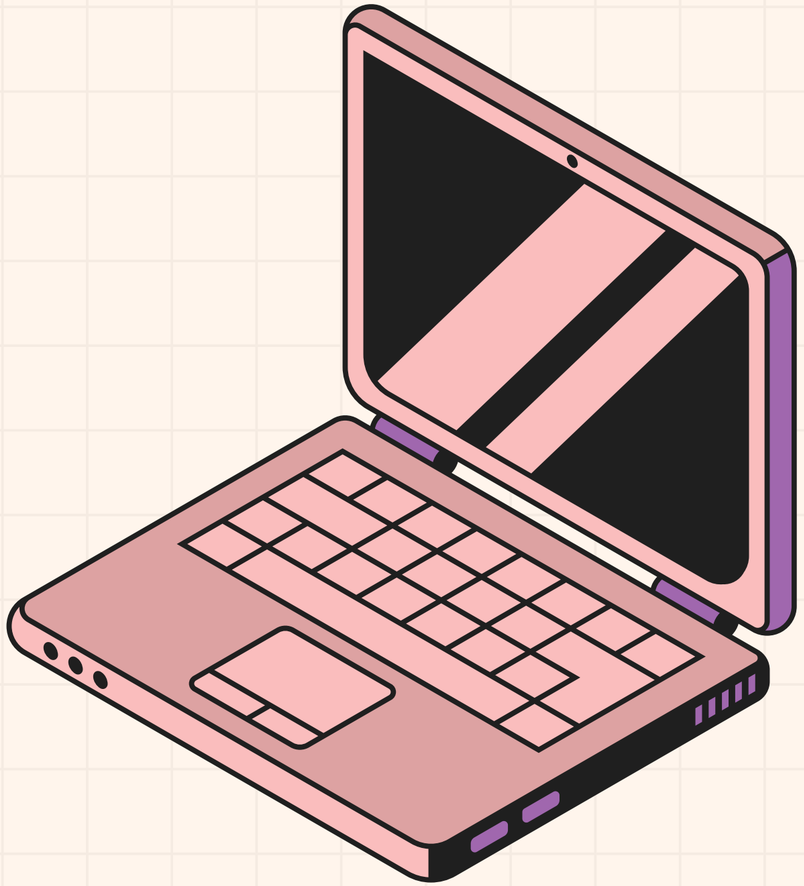
JHUNIOR FRANCO

MAICOL CORDOBA



TEMAS

- Que es Codigo Limpio
- Principios SOLID
- Arquitectura de la Cebolla
- Arquitectura Hexagonal
- Clean Architectur
- Relación entre Código Limpio y estas Arquitecturas
- Qué arquitecturas existen además?
- Preguntas

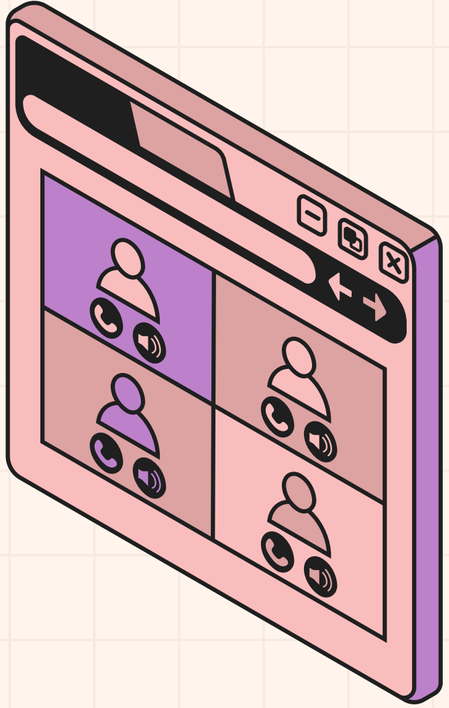


¿QUÉ ES CÓDIGO LIMPIO?

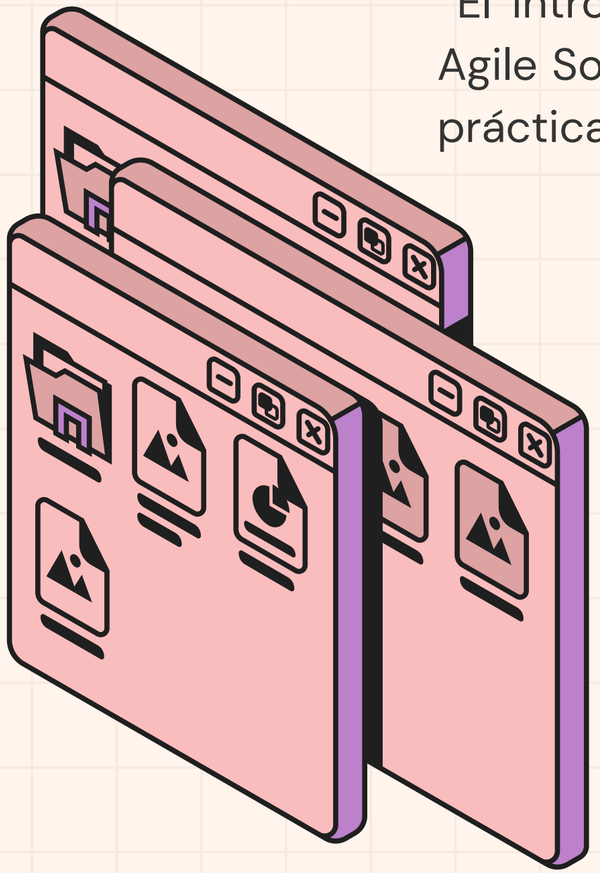
El Código Limpio es una forma de escribir software que sea fácil de leer, mantener, extender y probar.

Su objetivo es reducir la complejidad, evitar errores y garantizar que cualquier desarrollador pueda entenderlo rápidamente.

¿DE DÓNDE VIENE EL CÓDIGO LIMPIO?



El concepto de Código Limpio (Clean Code) proviene del trabajo del ingeniero de software Robert C. Martin, conocido como “Uncle Bob”.
Él introdujo formalmente la idea en su libro “Clean Code: A Handbook of Agile Software Craftsmanship” (2008), donde establece principios y buenas prácticas para escribir software profesional, legible y mantenible



Aunque Uncle Bob lo popularizó, sus bases vienen de metodologías más antiguas como:

- Programación estructurada (años 70)
- Buenas prácticas de ingeniería de software
- Metodologías ágiles como XP (Extreme Programming)
- Principios SOLID, también creados por él

PRINCIPIOS
SOLID

es un conjunto de principios que ayudan a escribir código limpio, escalable y mantenible.

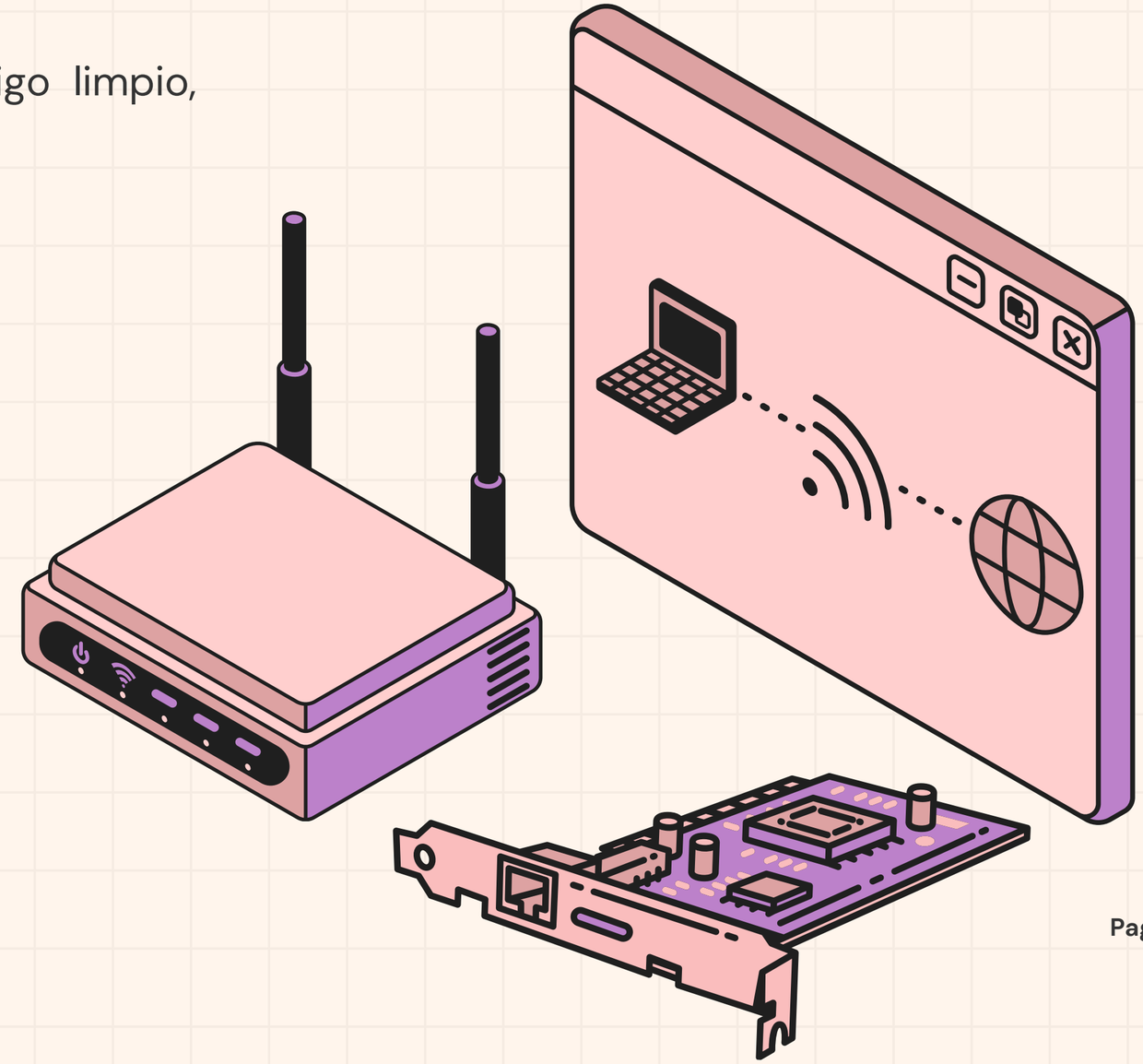
S — Responsabilidad Única
Cada clase o módulo debe hacer una sola cosa.

O — Abierto/Cerrado
El código debe permitir añadir nuevas funciones sin modificar lo existente.

L — Sustitución de Liskov
Las clases hijas deben poder usarse como sustitutas de las clases padre.

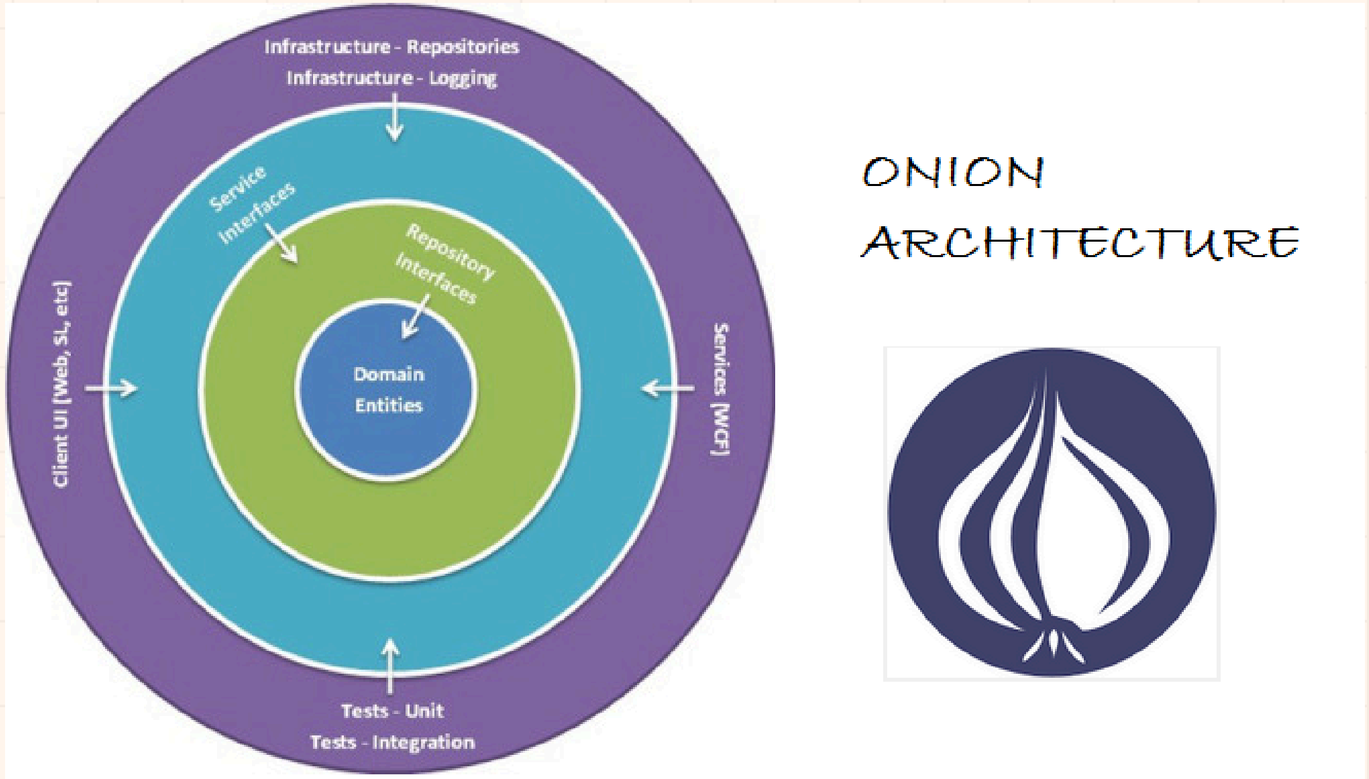
I — Segregación de Interfaces
Es mejor tener interfaces pequeñas y específicas que una grande y compleja.

D — Inversión de Dependencias
El código debe depender de abstracciones, no de detalles concretos.



ARQUITECTURA DE LA CEBOLLA

Arquitectura basada en capas concéntricas donde el Dominio es el centro del sistema.



Capas principales:

- Núcleo: Dominio / Entidades
- Capa de Aplicación
- Infraestructura
- Interfaces externas

Idea clave:
Las capas externas dependen de las internas, nunca al revés.

ARQUITECTURA HEXAGONAL

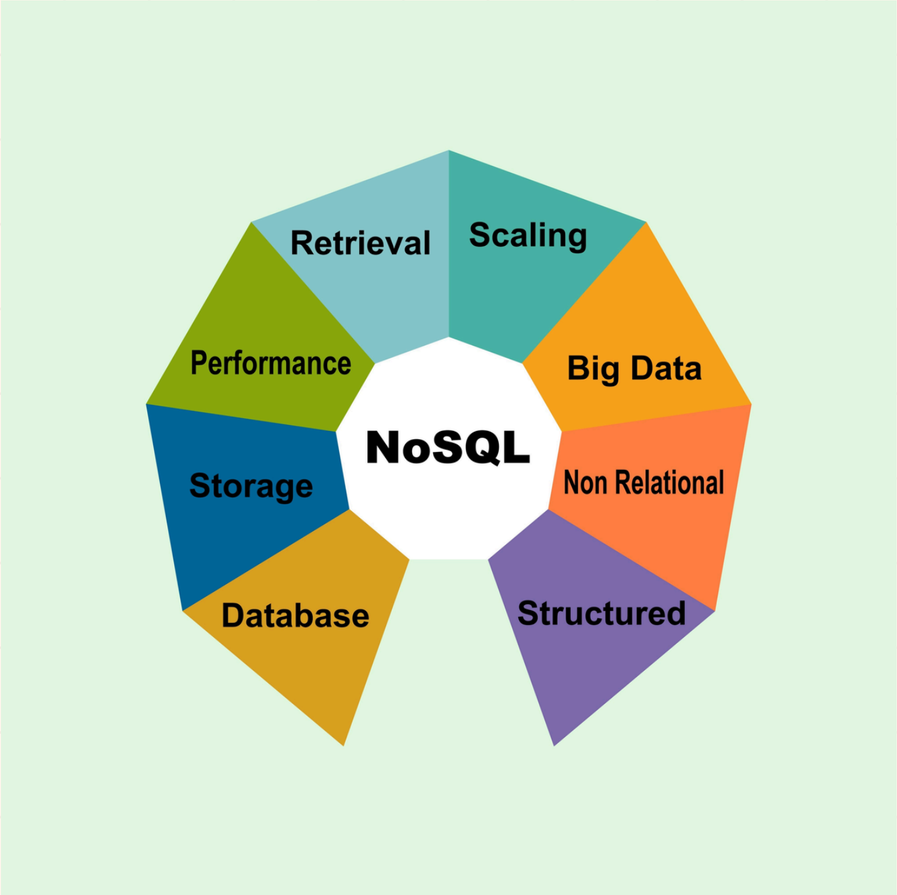
también conocida como Puertos y Adaptadores, es un patrón de diseño fundamental que busca aislar tu dominio o lógica de negocio de las preocupaciones técnicas externas.

Estructura:

- Dominio (núcleo de reglas)
- Puertos (interfaces que definen cómo interactuar)
- Adaptadores (implementaciones como DB, API, UI)

Objetivo:

Aislar completamente la lógica del negocio de las tecnologías externas.



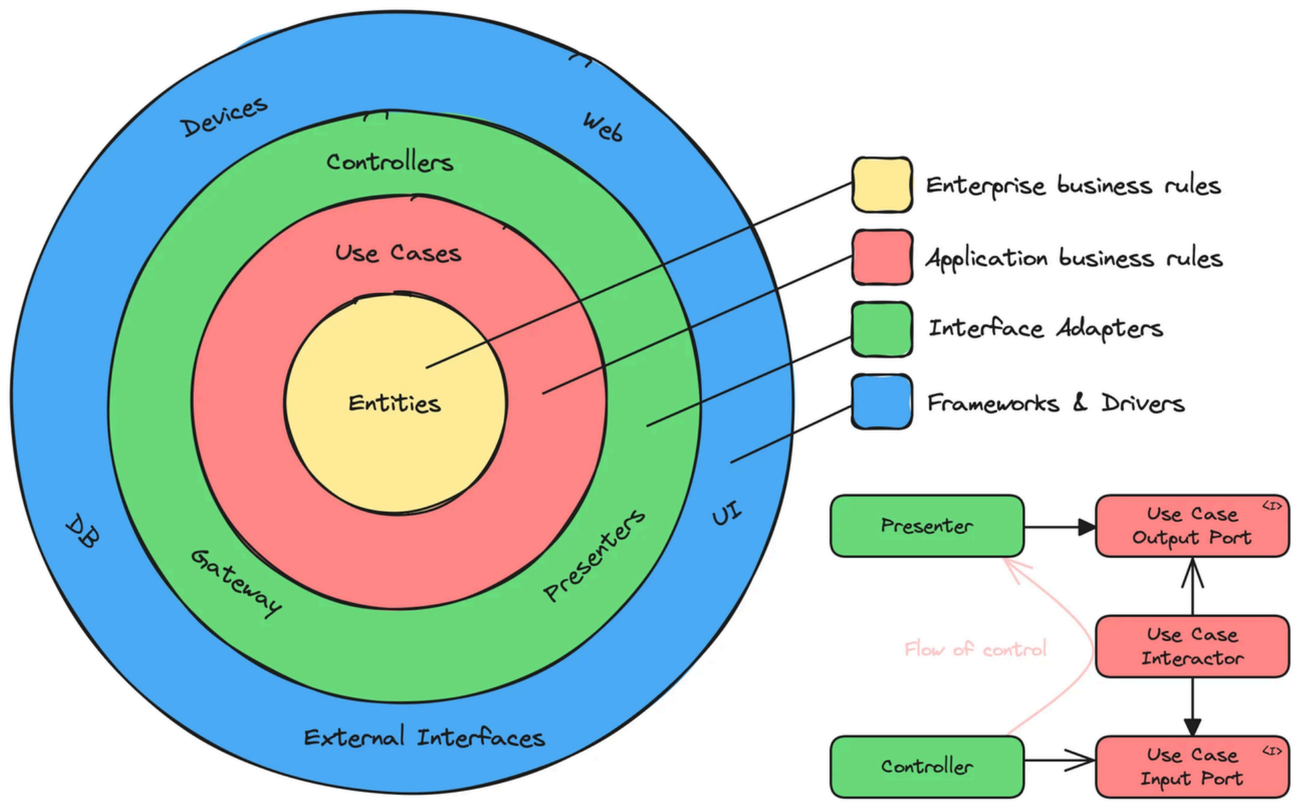
CLEAN ARCHITECTURE

Propuesta por Robert C. Martin.
Combina ideas de la Cebolla, Hexagonal y DDD.

Capas típicas:

- Entidades
- Casos de uso
- Controladores y Presentadores
- Infraestructura / Implementaciones

Regla principal:
Las dependencias apuntan hacia el dominio.



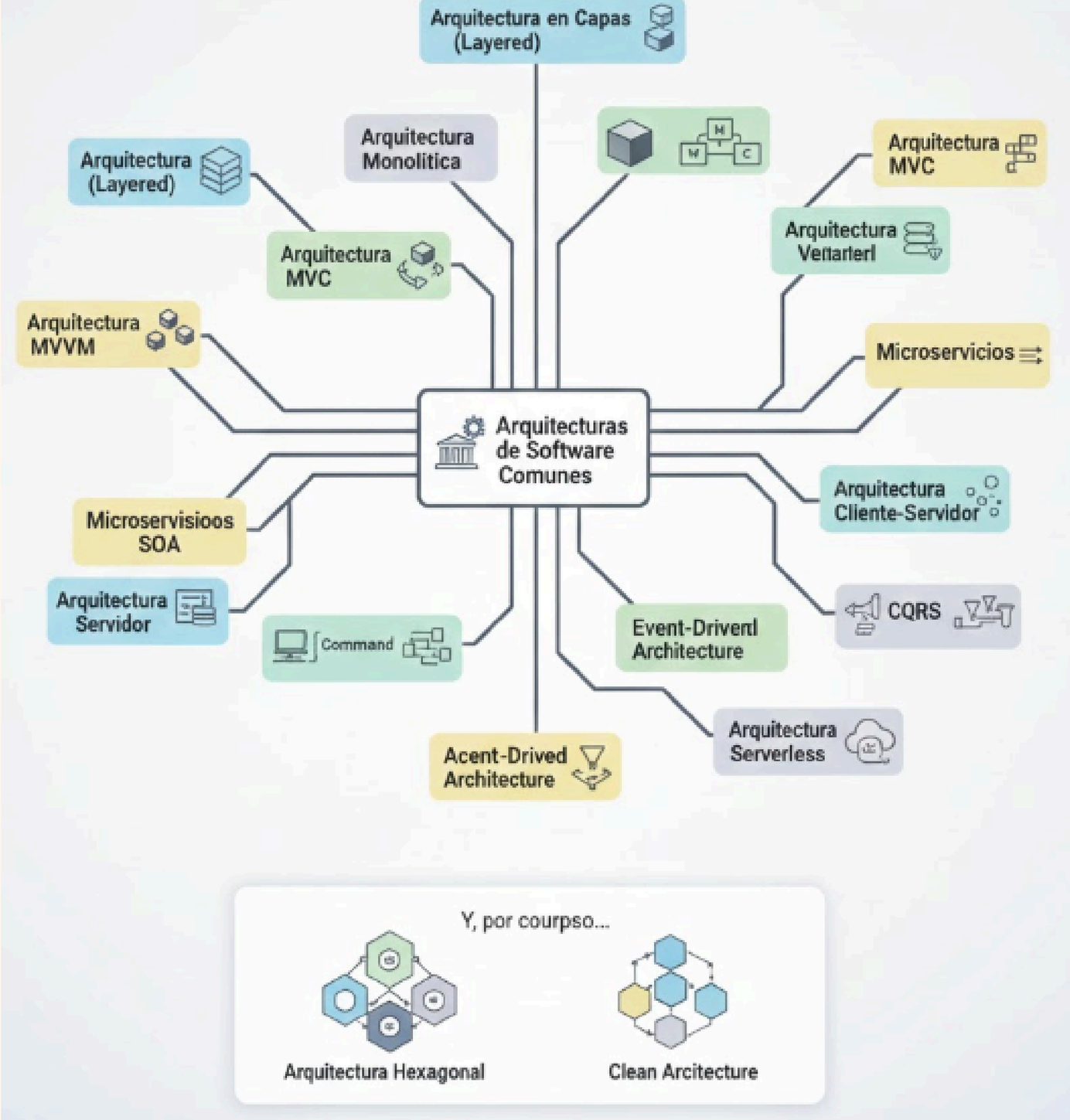
RELACIÓN ENTRE CÓDIGO LIMPIO Y ESTAS ARQUITECTURAS

- SOLID es la base del código limpio.
- Código limpio es la base para arquitecturas buenas.
- Cebolla, Hexagonal y Clean aplican SOLID a nivel de proyecto completo.
- Todas buscan lo mismo:
- sistemas mantenibles
- poco acoplados
- fáciles de probar
- organizados por responsabilidad y no por tecnología.

¿QUÉ ARQUITECTURAS EXISTEN ADEMÁS?

Estas son las arquitecturas más conocidas y utilizadas en el desarrollo de software:

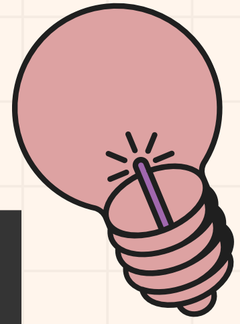
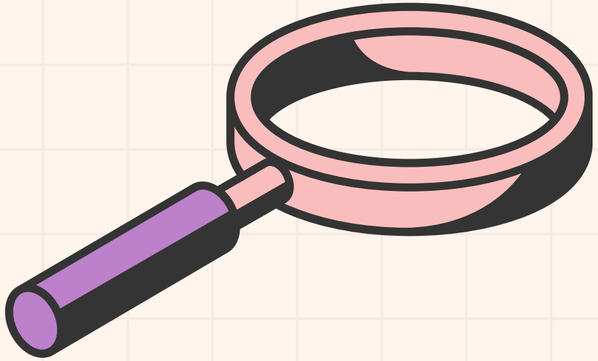
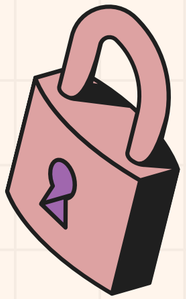
- Arquitectura en Capas (Layered)
- Arquitectura Monolítica
- Arquitectura MVC
- Arquitectura MVVM
- Arquitectura SOA
- Microservicios
- Event-Driven Architecture
- Arquitectura Cliente-Servidor
- Arquitectura Serverless
- CQRS
- Pipes & Filters



CONCLUSIÓN

El Código Limpio y los principios SOLID son la base para desarrollar software profesional.

Las arquitecturas Cebolla, Hexagonal y Clean aplican estos principios para crear sistemas flexibles, escalables y fáciles de mantener.

Usarlas correctamente garantiza calidad, estabilidad y crecimiento ordenado en los proyectos.

GRACIAS POR LA ATENCION